

LabConnect — Livrable du projet

Plateforme de vente et de prise de rendez-vous d'appel pour du matériel de laboratoire, composée d'une API backend (FastAPI + PostgreSQL) et d'une application frontend (React + Vite).

1. Accès au projet

Élément	Valeur
URL publique (frontend)	https://labpro-ten.vercel.app/
URL publique (API backend)	https://labpro.onrender.com
Dépôt Git (GitHub)	https://github.com/Mickael-C21/Labpro

⚠ Le backend est hébergé sur Render (plan gratuit) : le premier appel après une période d'inactivité peut prendre 30 à 60 secondes le temps que le service redémarre ("cold start"). C'est normal, ce n'est pas un bug.

2. Structure du dépôt

```
Labpro/
├── Call_center_API/           # Backend FastAPI (Python)
│   ├── main.py               # Routes de l'API
│   ├── model.py              # Modèles SQLAlchemy (tables)
│   ├── schema.py             # Schémas Pydantic (validation)
│   ├── auth.py               # Authentification JWT + hash mots de passe
│   ├── database.py           # Connexion PostgreSQL
│   ├── seed_products.py      # Données de démo (catalogue produits)
│   ├── reset.py              # Script utilitaire : vide la table produits
│   ├── requirements.txt      # Dépendances Python
│   ├── .env.example          # Modèle de configuration backend
│   └── sql_export/
│       └── labconnect_export.sql # Export SQL (schéma + données de démo)
├── Front-figma/              # Frontend React / Vite
│   ├── src/                  # Code source de l'application
│   ├── package.json          # Dépendances Node.js
│   ├── vercel.json           # Config de déploiement Vercel (routing SPA)
│   └── .env.example           # Modèle de configuration frontend
└── run_backend.bat           # Lance le backend (Windows)
```

```
└─ start_backend.bat          # Lance le backend avec rechargement auto (Windows)
└─ README.md                  # Ce fichier
```

3. Prérequis d'installation

- **Python** 3.11 ou supérieur
 - **Node.js** 18 ou supérieur (+ npm, ou pnpm en alternative)
 - **PostgreSQL** 14 ou supérieur (local ou hébergé, ex. Render/Neon/Supabase)
 - **Git**
-

4. Installation en local

4.1 Cloner le dépôt

```
git clone https://github.com/Mickael-C21/Labpro.git
cd Labpro
```

4.2 Backend (Call_center_API)

```
cd Call_center_API
python -m venv venv

# Windows
venv\Scripts\activate
# macOS / Linux
source venv/bin/activate

pip install -r requirements.txt
```

Créer le fichier de configuration à partir du modèle fourni :

```
cp .env.example .env      # macOS / Linux
copy .env.example .env    # Windows
```

Puis éditer `.env` et renseigner :

```
DATABASE_URL=postgresql://<utilisateur>:<mot_de_passe>@<hote>:5432/<nom_base>
SECRET_KEY=<une valeur aléatoire forte>
```

Générer une clé secrète aléatoire si besoin :

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Initialiser la base de données — deux options :

- **Option A (recommandée pour retrouver l'état de démo tel quel)** : importer le dump SQL fourni (voir section 5).
- **Option B (automatique)** : au premier démarrage, l'API crée elle-même les tables (`model.Base.metadata.create_all`) et insère les données de démo (agents, compte admin, catalogue produits) via les fonctions `ensure_default_*` de `main.py` . Il suffit donc de démarrer l'API sur une base vide.

Lancer le serveur :

```
python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

L'API est alors disponible sur `http://127.0.0.1:8000` (documentation interactive auto-générée sur `http://127.0.0.1:8000/docs`).

Sous Windows, les scripts `start_backend.bat` / `run_backend.bat` à la racine du dépôt automatisent ces étapes (adapter le chemin du projet et de l'environnement virtuel dans le script si besoin).

4.3 Frontend (Front-figma)

Dans un second terminal :

```
cd Front-figma  
npm install
```

Créer le fichier de configuration :

```
cp .env.example .env      # macOS / Linux  
copy .env.example .env    # Windows
```

Par défaut, `VITE_API_URL` pointe vers `http://127.0.0.1:8000` (backend local). Pour utiliser l'API déployée sur Render au lieu du backend local, mettre :

```
VITE_API_URL=https://labpro.onrender.com
```

Lancer le serveur de développement :

```
npm run dev
```

Le site est alors disponible sur `http://localhost:5173` .

Build de production :

```
npm run build
```

5. Fichier d'export SQL

Le fichier `Call_center_API/sql_export/labconnect_export.sql` contient :

- Le schéma complet (`CREATE TABLE`) des 4 tables : `users` , `products` , `agents` , `calls` — généré à partir des modèles SQLAlchemy (`model.py`).
- Les données de démo : les 2 agents par défaut, le compte administrateur de test, un compte client de test, et les 20 produits du catalogue de démonstration.

Pour l'importer dans une base PostgreSQL vide :

```
psql "postgresql://<utilisateur>:<mot_de_passe>@<hote>:5432/<nom_base>" \  
-f Call_center_API/sql_export/labconnect_export.sql
```

6. Identifiants de connexion à la base de données

Les identifiants de connexion à la base PostgreSQL de production (hébergée sur Render) sont des informations sensibles et **ne sont pas inclus dans ce livrable** (ni dans le dépôt Git, ni dans ce document), conformément aux bonnes pratiques de sécurité — ils ne doivent jamais être committés en clair.

Pour obtenir un accès à la base de production, contacter : tsoumboumickael19@gmail.com.

Pour un usage local/test, créer votre propre base PostgreSQL et renseigner ses identifiants dans `Call_center_API/.env` (voir section 4.2). Le fichier `.env.example` documente le format attendu.

7. Identifiants de test

7.1 Espace client

- Créer un compte via la page **Inscription** (`/register`), ou
- Utiliser le compte de démonstration créé par le dump SQL :
 - Email : `client@test.fr`
 - Mot de passe : `client123`

7.2 Accès administrateur (back-office)

Le back-office est accessible depuis l'URL publique via l'onglet **Espace Administrateur** :

- Page de connexion : <https://labpro-ten.vercel.app/admin>
- Tableau de bord (après connexion) : <https://labpro-ten.vercel.app/admin/dashboard>

Identifiants :

- Email : `admin@test.fr`
- Mot de passe : `admin123`

Ce compte est créé automatiquement au premier démarrage de l'API s'il n'existe aucun administrateur en base (voir `ensure_default_admin` dans `main.py`), et il est également inclus dans le dump SQL fourni.

Depuis le tableau de bord, l'administrateur peut consulter l'ensemble des demandes d'appel (`GET /calls`), les utilisateurs (`GET /users`), et mettre à jour le statut / les notes d'un rendez-vous.

8. Vérification de la compatibilité multi-navigateur

Avant la livraison, vérifier le bon fonctionnement du site (navigation, inscription/connexion, connexion admin, demande de rendez-vous, suivi des rendez-vous, chatbot) sur au moins les navigateurs suivants :

- **Google Chrome** (dernière version)
- **Mozilla Firefox** (dernière version)
- **Microsoft Edge** (dernière version)
- **Safari** (macOS / iOS, si disponible)

Points d'attention particuliers :

- L'API autorise les requêtes cross-origin depuis n'importe quelle origine (`CORS allow_origins=["*"]` dans `main.py`), ce qui évite les blocages CORS différents d'un navigateur à l'autre.
- Le token de session est stocké dans le `localStorage` du navigateur : se déconnecter/reconnecter permet de vérifier que la session persiste bien après un rafraîchissement de page, sur chaque navigateur testé.
- Vérifier l'affichage responsive (mobile/tablette/desktop) via les outils de développement de chaque navigateur.

9. Support

Pour toute question concernant l'installation ou l'accès au projet : tsoumboumickael19@gmail.com.